Event Handling in React

6 min read · Updated 16 Jul 2026 · Topic 8 of 62

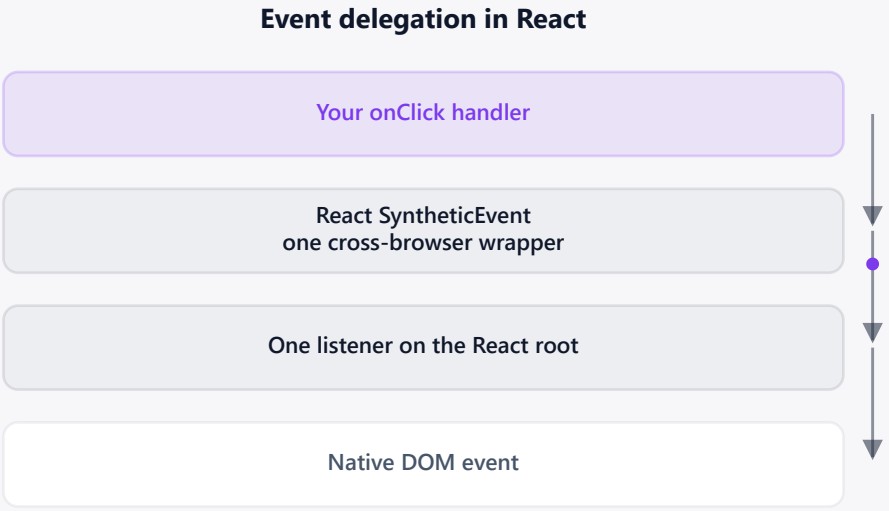
Bookmark · Mark done

1.8 — Event Handling in React

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"How does event handling work in React?"	🟢	★★★★★
"What are Synthetic Events?"	🟡	★★★★☆
"How do you pass arguments to an event handler?"	🟢	★★★★☆
"What is event delegation? Does React use it?"	🟡	★★★☆☆
"What is the difference between `onClick={fn}` and `onClick={fn()}`?"	🟢	★★★★★
"How do you prevent default behavior?"	🟢	★★★☆☆

How Events Work in React



React attaches ONE listener at the root and routes native events to your handlers as SyntheticEvents.

React's event system looks like regular HTML events but works differently under the hood. You write `onClick` instead of `onclick`, and you pass a function reference instead of a string.

JSX Copy

```jsx
// HTML (plain)
<button onclick="handleClick()">Click</button>

// React (JSX)
<button onClick={handleClick}>Click</button>
```

Three differences to remember:

1. **CamelCase naming:** `onClick`, `onChange`, `onSubmit`, `onKeyDown` (not `onclick`, `onchange`)

3 **Prevent default explicitly:** You call `e.preventDefault()` inside the handler. Returning `false` does nothing in React (unlike vanilla HTML).

Passing Arguments to Event Handlers

This is a very common interview question because there's a trap here.

JSX Copy

```jsx
// ❌ WRONG: calls the function immediately on render, not on click
<button onClick={handleDelete(item.id)}>Delete</button>

// ✅ CORRECT: wraps in an arrow function, calls on click
<button onClick={() => handleDelete(item.id)}>Delete</button>

// ✅ ALSO CORRECT: bind the argument
<button onClick={handleDelete.bind(null, item.id)}>Delete</button>
```

When you write `handleDelete(item.id)` without wrapping it, JavaScript executes it right away during render and passes the *return value* to onClick. Unless your handler returns another function, this is a bug.

The Event Object

Every handler automatically receives the event object as its first argument.

JSX Copy

```jsx
function SearchInput() {
  const handleKeyDown = (e) => {
    if (e.key === 'Enter') {
      console.log('User pressed Enter');
    }
  };
```

```
  return (
    <input
      onChange={handleChange}
      onKeyDown={handleKeyDown}
      placeholder="Search ... "
    />
  );
}
```

When you need to pass both custom arguments AND the event object:

JSX Copy

```jsx
const handleClick = (id, e) => {
  e.stopPropagation();
  console.log('Clicked item:', id);
};

return <button onClick={(e) => handleClick(item.id, e)}>Click</button>;
```

Synthetic Events



SyntheticEvent normalizes browser differences so your handler code stays the same.

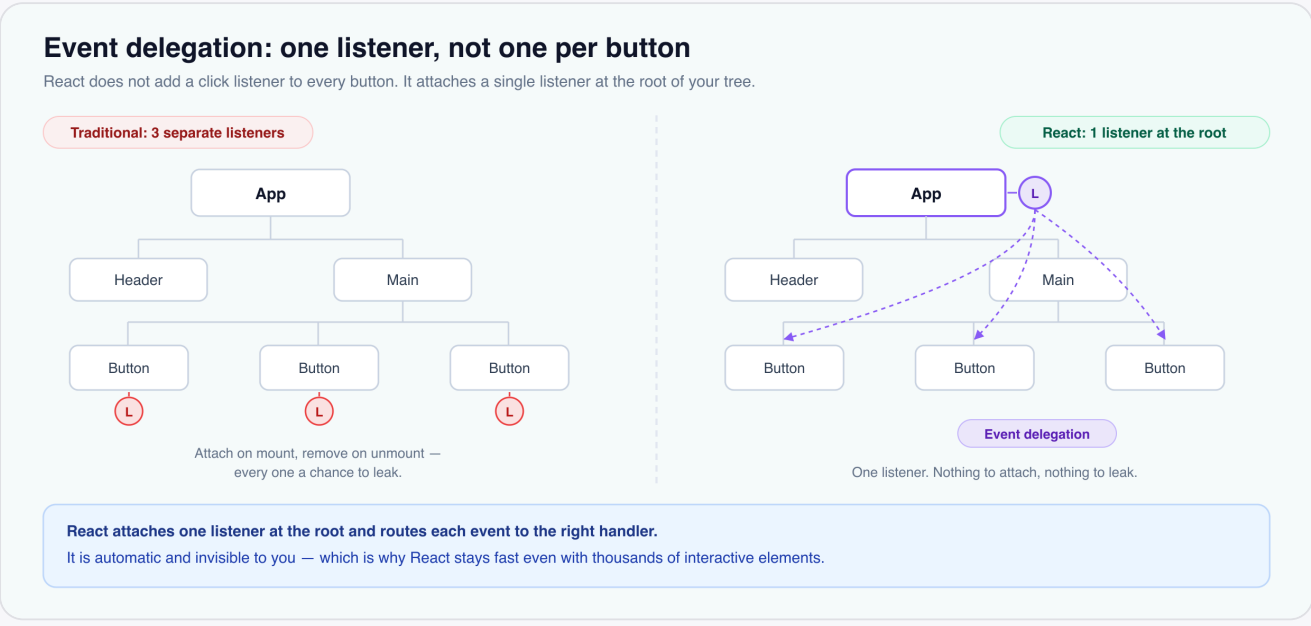
React doesn't attach event listeners directly to DOM nodes. Instead, it uses its own event system called **Synthetic Events**.

A SyntheticEvent is a cross-browser wrapper around the browser's native event. It has the same interface as native events (`stopPropagation()` , `preventDefault()` , `target` , `currentTarget`) but works identically across all browsers.

Feature	Native Events	React Synthetic Events
Browser consistency	Varies by browser	Identical across all browsers
Where listeners attach	Directly on elements	Single listener on the root (delegation)
Event object	Browser-specific	Normalized wrapper
Performance	One listener per element	One listener total (pooled)

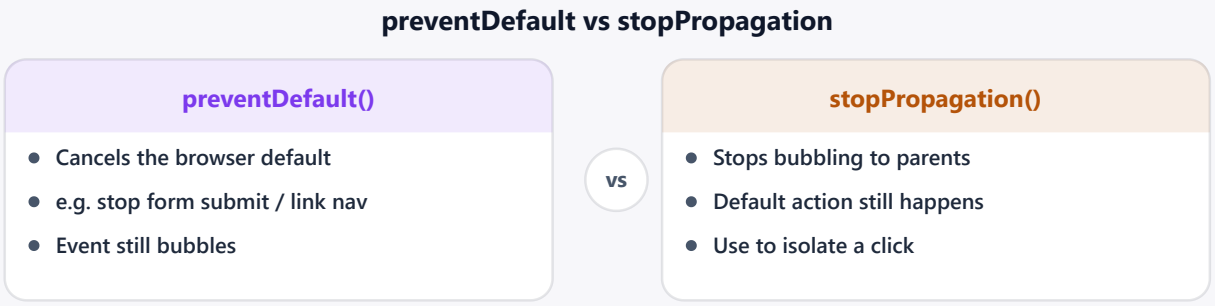
Event Delegation in React

React doesn't add a click listener to every button in your app. It attaches a single event listener at the root of your React tree and uses event delegation to figure out which component should handle each event.



interactive elements.

Preventing Default Behavior



They solve different problems — one cancels the default action, the other stops the event travelling up.

In vanilla HTML, you can return `false` from an event handler to prevent defaults. In React, that doesn't work. You must call `e.preventDefault()` explicitly.

JSX Copy

```jsx
function LoginForm() {
  const handleSubmit = (e) => {
    e.preventDefault();  // Prevents page reload
    // Handle form submission
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="email" />
      <button type="submit">Login</button>
    </form>
  );
}
```

Stopping Propagation

To prevent an event from bubbling up to parent handlers:

```
function Card() {
  const handleCardClick = () => console.log('Card clicked');

  const handleButtonClick = (e) => {
    e.stopPropagation();  // Prevents handleCardClick from firing
    console.log('Button clicked');
  };

  return (
    <div onClick={handleCardClick}>
      <h2>Card Title</h2>
      <button onClick={handleButtonClick}>Action</button>
    </div>
  );
}
```

Without `stopPropagation`, clicking the button would fire both `handleButtonClick` AND `handleCardClick` because events bubble up from child to parent.

Common Event Types Quick Reference

Event	Handler	Common Use
Click	`onClick`	Buttons, links, toggles
Input change	`onChange`	Text inputs, selects, checkboxes
Form submit	`onSubmit`	Form submission
Key press	`onKeyDown` / `onKeyUp`	Keyboard shortcuts, Enter to submit

Mouse hover	`onMouseEnter` / `onMouseLeave`	Tooltips, dropdowns
Scroll	`onScroll`	Infinite scroll, scroll-based animations

Common Mistakes

Mistake 1: Calling the function instead of passing it.

JSX Copy

```jsx
// ❌ Executes immediately. Infinite loop if it sets state.
<button onClick={setCount(count + 1)}>+</button>

// ✅ Pass a function that React will call on click
<button onClick={() => setCount(count + 1)}>+</button>
```

Mistake 2: Forgetting that `onChange` fires on every keystroke in React. In vanilla HTML, `onchange` fires on blur. React's `onChange` fires on every character typed. This is intentional and different from HTML.

Mistake 3: Using `onKeyPress` instead of `onKeyDown`. `onKeyPress` is deprecated. Use `onKeyDown` for keyboard events.

Output Question

JSX Copy

```jsx
function App() {
  const handleClick = () => console.log('Button');
```

```
    <div onClick={handleDivClick}>
      <button onClick={handleClick}>Click me</button>
    </div>
  );
}
```

What logs when you click the button?

Answer: Both `"Button"` and `"Div"` log, in that order. Events bubble up from the button to the div. To prevent the div handler from firing, add `e.stopPropagation()` inside `handleClick`.

🗣️ Interview Answer

> "React uses a Synthetic Event system that wraps native browser events to ensure consistent behavior across browsers. Events use camelCase naming like onClick and onChange, and you pass function references rather than strings. Under the hood, React uses event delegation by attaching a single listener at the root rather than one per element, which is more performant. You prevent default behavior with `e.preventDefault()` and stop bubbling with `e.stopPropagation()`."

✅ Key Takeaways

- React uses **camelCase** for events: `onClick`, `onChange`, `onSubmit`.
- Pass a **function reference**, not a function call: `onClick={fn}` not `onClick={fn()}`.
- **Synthetic Events** are cross-browser wrappers around native events.
- React uses **event delegation** at the root, not individual listeners per element.
- Use `e.preventDefault()` to prevent defaults. Returning `false` does nothing.

- Use `e.stopPropagation()` to stop event bubbling.
- `onChange` in React fires on **every keystroke**, unlike HTML's `onchange` which fires on blur.
- `onKeyPress` is **deprecated**. Use `onKeyDown`.

QUICK CHECK — no marks, just for you.

1. In a React component you write <button onClick={handleDelete(item.id)}>Delete</button>. What actually happens?

- ○ handleDelete runs on click, but the event object replaces item.id as the first argument
- ○ handleDelete runs only when the button is clicked, with item.id as its argument
- ○ React throws an error because event handlers may not receive custom arguments
- ○ handleDelete runs immediately during render, and its return value is what gets passed to onClick

Check answer

2. According to the lesson, where does React attach the DOM event listeners for handlers like onClick?

- ○ A single listener at the root of the React tree, which delegates events to the right handler
- ○ One listener per component instance, attached when the component mounts
- ○ One listener directly on each DOM element that has an onClick prop
- ○ No real DOM listeners at all; React polls the browser for user interactions

Check answer

3. ...from plain HTML to React and returns false from the submit handler to stop the page reloading, and also expects onChange on the text input to fire only when the field loses focus. What does the lesson say will actually happen?

- ○ The page reload is prevented as in HTML, but onChange fires on every keystroke
- ○ The page still reloads because returning false does nothing, and onChange correctly fires only on blur
- ○ The page still reloads because returning false does nothing in React, and onChange fires on every keystroke
- ○ Both behaviours match plain HTML, since Synthetic Events replicate native event semantics exactly

Check answer

‹ Previous
Forms: Controlled vs Uncontrolled Components

Next ›
Refs & useRef: Beyond DOM Access

LEARN MORE

ARTICLES & DOCS

- Responding to Events
  react.dev
- Handling Events (deep dive)
  reactjs.org
- React event object (SyntheticEvent)
  react.dev

VIDEOS



React.js Interview Kit



Event handling, synthetic events & bubbling

YouTube



Synthetic events in React

YouTube

Hand-picked references for this lesson.